

Instituto Tecnológico de Buenos Aires

Bases de Datos II

Trabajo Práctico Obligatorio

VetSalud

Grupo 7

Integrantes:

- Rodriguez Acevedo, Clara - 66527
- Buela, Mateo - 64680
- Medina, Sol - 63577

Fecha de entrega: 12 junio 2026

Índice

1. Introducción	1
2. Decisiones de Bases de Datos	2
2.1. Elección de MongoDB	2
2.2. Elección de Redis	2
2.3. División de responsabilidades entre motores	3
2.4. Conexión desde la aplicación	4
3. Modelo de Datos	5
3.1. Modelo en MongoDB	5
3.2. Modelo en Redis	8
3.3. Relación entre ambos motores	10
4. Implementación de las Queries	11
4.1. Queries resueltas con MongoDB	11
4.1.1. Query 1: Pacientes activos con datos de propietario	11
4.1.2. Query 2: Consultas en seguimiento con veterinario y costo	12
4.1.3. Query 3: Historial completo de un paciente	13
4.1.4. Query 4: Propietarios con más de un paciente	15
4.1.5. Query 6: Pacientes con vacunas vencidas	17
4.1.6. Query 9: Consultas de control con costo menor a \$5.000	18
4.1.7. Query 10: Pacientes de una sucursal determinada	19
4.1.8. Query 12: Propietarios a revisar	20
4.1.9. Query 13: ABM de propietarios	21
4.2. Queries resueltas con Redis	22
4.2.1. Query 15: Decremento de stock tras una consulta	22
4.3. Queries que combinan MongoDB y Redis	23
4.3.1. Query 8: Stock de productos con menos de 50 unidades	23
4.3.2. Query 14: Registro de nueva consulta médica	24
4.4. Queries cacheadas	25
4.4.1. Query 5: Veterinarios activos con consultas en los últimos 60 días	26
4.4.2. Query 7: Top 5 diagnósticos más frecuentes	27
4.4.3. Query 11: Ingresos por veterinario en el mes actual	28

5. Manual de Usuario	30
5.1. Requisitos previos	30
5.2. Primera ejecución	30
5.3. Carga inicial de datos	31
5.4. Uso del dashboard	31
5.5. Consultas disponibles desde la interfaz	32
5.6. ABM de propietarios	32
5.7. Registro de consultas	33
5.8. Manejo de stock	34
5.9. Uso directo de la API	35
5.10. Caché	36
6. Resultados	37
7. Limitaciones	38
8. Conclusiones	39
9. Referencias	40

1 Introducción

VetSalud es una red ficticia de clínicas veterinarias en Argentina. Este trabajo busca montar un sistema sobre una arquitectura de persistencia polígota con dos motores NoSQL. El sistema gestiona propietarios, sus mascotas o pacientes, veterinarios, consultas y cirugías, vacunaciones, y el stock farmacéutico. Las bases corren en contenedores Docker, y construimos una API REST en Node con Express junto con un dashboard web para ejecutar y visualizar las consultas.

La elección de los dos motores respondió al perfil de los datos. Todo lo clínico y administrativo, es decir propietarios, pacientes, veterinarios, consultas, vacunaciones y catálogo de productos, vive en **MongoDB**. Son datos con esquemas que podrían cambiar con el tiempo y queríamos evitar que migrar fuera dificultoso. Hoy un paciente tiene nombre, especie y raza; mañana podríamos sumar peso, alergias o antecedentes médicos. En MongoDB realizamos consultas analíticas que cruzan entidades, filtran por fecha y agrupan por categoría, usando pipelines del Aggregation Framework con etapas como `$match`, `$lookup`, `$group` y `$project`.

Por otra parte, utilizamos **Redis** para administrar las unidades disponibles del stock farmacéutico. Las unidades de cada producto cambian con cada consulta que consume stock, por lo que necesitábamos operaciones rápidas y atómicas sobre ese contador: leer cuántas unidades hay de un producto, listar productos con poco stock y descontar unidades reduciendo el riesgo de que dos consultas concurrentes dejen el contador en negativo. Redis también cumple un rol de caché de resultados de queries pesadas. Algunas consultas del dashboard agregan sobre toda la colección de consultas y pueden leerse seguido; cachearlas en Redis con TTL corto evita ejecutar la misma agregación todo el tiempo.

El objetivo del trabajo no fue solamente implementar las consultas pedidas, sino también justificar cómo se distribuyen las responsabilidades entre ambos motores y qué ventajas aporta cada uno dentro del sistema. MongoDB concentra el dominio persistente y estructurado de la clínica, mientras que Redis se reserva para información de acceso rápido, contadores y resultados derivados.

2 Decisiones de Bases de Datos

2.1 Elección de MongoDB

Elegimos MongoDB como base principal del dominio clínico y administrativo del sistema. La decisión se apoyó en las características del dominio y en el tipo de consultas que debíamos resolver.

- **Flexibilidad de esquema:** el dominio veterinario puede cambiar con el tiempo. Hoy un paciente tiene nombre, especie, raza y fecha de nacimiento; en una versión futura podría incorporar peso, alergias, antecedentes o alertas médicas. MongoDB permite agregar estos campos sin rediseñar todo el esquema.
- **Datos con estructura de documento:** las entidades principales del sistema tienen varios atributos propios y se representan naturalmente como documentos. Propietarios, pacientes, veterinarios, consultas, vacunaciones y productos no son simples pares clave-valor, sino registros con estructura rica.
- **Consultas analíticas sobre el dominio:** muchas queries requieren filtrar, cruzar entidades, agrupar y proyectar resultados. Para esto usamos pipelines del Aggregation Framework con etapas como `$match`, `$lookup`, `$group`, `$project` y `$sort`.
- **Atomicidad por documento:** las operaciones principales del sistema afectan un documento central por vez, como el alta de un propietario, la modificación de sus datos, la baja lógica de un propietario o la inserción de una consulta. MongoDB garantiza atomicidad sobre cada documento, lo cual alcanza para la mayoría de las operaciones del dominio clínico.
- **Consistencia para datos clínicos y administrativos:** en un backoffice clínico es importante no perder ni duplicar registros. Por eso usamos MongoDB como fuente principal del dominio persistente y centralizamos allí las escrituras de propietarios, pacientes, veterinarios, consultas, vacunaciones y productos.

2.2 Elección de Redis

Elegimos Redis para resolver las partes del sistema que necesitaban acceso rápido, operaciones atómicas simples y expiración automática. En nuestro caso, Redis cumple dos roles: manejar las unidades disponibles del stock farmacéutico y cachear resultados de consultas pesadas.

- **Stock como contador frecuente:** las unidades disponibles de cada producto cambian cada vez que una consulta consume insumos. Este dato se comporta más como

un contador operativo que como un documento clínico, por lo que Redis resulta más adecuado que MongoDB para administrarlo.

- **Lectura rápida de stock puntual:** la lógica de stock necesita consultar cuántas unidades quedan de un producto específico. Redis permite resolver esta lectura de forma directa y con muy baja latencia.
- **Consulta eficiente de stock bajo:** también necesitamos listar productos cuyo stock esté por debajo de cierto umbral. Para eso usamos un *Sorted Set*, donde el identificador del producto es el valor y la cantidad disponible es el score.
- **Decremento atómico:** cuando una consulta consume productos, el decremento individual del contador debe ejecutarse de forma indivisible para evitar resultados negativos. Redis permite validar el stock disponible y descontar unidades de forma atómica mediante un script Lua.
- **Caché con TTL:** algunas queries del dashboard agregan sobre toda la colección de consultas y pueden ejecutarse muchas veces. Redis permite guardar estos resultados con un TTL corto, evitando recalcular agregaciones costosas en cada lectura.

2.3 División de responsabilidades entre motores

La arquitectura quedó dividida según el tipo de dato y el tipo de operación. MongoDB concentra la información persistente y descriptiva del sistema, mientras que Redis almacena datos operativos o derivados que requieren acceso rápido.

- **MongoDB como fuente principal del dominio:** propietarios, pacientes, veterinarios, consultas, vacunaciones y productos viven en MongoDB. Estos datos representan el estado clínico y administrativo de VetSalud.
- **Redis como fuente de verdad del stock:** las unidades disponibles de cada producto viven en Redis. El catálogo del producto está en MongoDB, pero la cantidad disponible se administra en Redis porque cambia con más frecuencia y requiere operaciones atómicas.
- **Redis como caché descartable:** las claves `cache:*` no contienen información primaria, sino resultados derivados de consultas sobre MongoDB. Si una clave expira o se invalida, el resultado puede recalcularse.
- **Separación por prefijos:** las claves de stock usan el prefijo `stock:`, mientras que las claves de caché usan `cache:`. Esto evita mezclar datos con semánticas distintas dentro del mismo motor.

- **Limitación cross-motor:** MongoDB y Redis no comparten una transacción global. Por eso, en operaciones que involucran ambos motores, como registrar una consulta y descontar stock, existe una posible ventana de inconsistencia. Para reducirla, primero se valida stock en Redis, luego se inserta la consulta en MongoDB y finalmente se descuenta el stock de forma atómica.

2.4 Conexión desde la aplicación

Ambos motores se levantan con Docker Compose. El archivo `docker-compose.yml` define un contenedor de MongoDB 7 y otro de Redis 7, exponiendo los puertos `27017` y `6379`. La aplicación Node.js se conecta a esos servicios mediante variables de entorno: `MONGO_URI`, `DB_NAME` y `REDIS_URL`.

La conexión se centraliza en el módulo `src/db.js`. Ese módulo expone una función `connect()` que inicializa los clientes la primera vez que se la invoca y luego reutiliza las mismas instancias para los siguientes requests. Para MongoDB se usa el cliente oficial `mongodb`, y para Redis se usa el cliente oficial `redis`.

Esta decisión evita abrir una conexión nueva por cada endpoint ejecutado. No se implementó un pool propio porque el driver de MongoDB ya administra internamente sus conexiones, mientras que Redis opera naturalmente como un servidor single-threaded que procesa comandos de forma secuencial. Para el alcance del trabajo, centralizar la conexión en un único módulo también simplifica el seed, las queries y el cierre ordenado de clientes.

3 Modelo de Datos

El modelo de datos se diseñó pensando en las consultas que debía resolver el sistema. La idea no fue copiar un modelo relacional tradicional dentro de una base NoSQL, sino repartir la información según el tipo de dato y el tipo de operación esperada. MongoDB concentra las entidades clínicas y administrativas, mientras que Redis guarda estructuras específicas para stock y caché.

El sistema utiliza la base MongoDB `vetsalud`, salvo que se configure otro nombre mediante variables de entorno. Durante la carga inicial, el script de seed lee los archivos CSV, transforma los tipos de datos necesarios y carga las colecciones desde cero. Los identificadores provistos por los datasets se conservan como `_id` de cada documento.

3.1 Modelo en MongoDB

En MongoDB se guardan las entidades principales del dominio de VetSalud: propietarios, pacientes, veterinarios, consultas, vacunaciones y productos. Cada una se representa como una colección independiente porque tiene identidad propia y puede ser consultada desde distintos puntos del sistema.

Las colecciones principales son:

- `propietarios`
- `pacientes`
- `veterinarios`
- `consultas`
- `vacunaciones`
- `productos`

La decisión general fue mantener las relaciones mediante referencias por id. Por ejemplo, un paciente referencia a su propietario con `id_propietario`, y una consulta referencia tanto al paciente como al veterinario mediante `id_paciente` e `id_vet`. Esto permite mantener las entidades separadas y evitar duplicar información descriptiva, pero al mismo tiempo deja disponible la posibilidad de cruzarlas mediante `$lookup` cuando una query necesita información combinada.

Los propietarios contienen la información de contacto y ubicación de cada cliente de la clínica. Además, incluyen un campo `activo`, utilizado para implementar baja lógica. Esta decisión permite conservar los datos históricos aunque un propietario deje de estar activo.

Ejemplo de propietario

```
1 {
2   "_id": "C001",
3   "nombre": "Ana",
4   "apellido": "Rodriguez",
5   "dni": "30456789",
6   "email": "ana@gmail.com",
7   "telefono": "1145001234",
8   "ciudad": "Buenos Aires",
9   "provincia": "Buenos Aires",
10  "activo": true
11 }
```

Los pacientes representan las mascotas atendidas por VetSalud. Cada paciente referencia a su propietario, pero no se encuentra embebido dentro del documento del dueño. Elegimos esta separación porque un propietario puede tener varios pacientes y porque muchas consultas necesitan trabajar directamente sobre la colección de pacientes. El campo `fecha_nac` se carga como fecha en MongoDB.

Ejemplo de paciente

```
1 {
2   "_id": "P001",
3   "nombre": "Firulais",
4   "especie": "Perro",
5   "raza": "Labrador",
6   "fecha_nac": "2019-04-12T00:00:00.000Z",
7   "id_propietario": "C001",
8   "activo": true
9 }
```

Los veterinarios se modelan como una colección propia porque son una entidad central para varias consultas del sistema. Además de los datos profesionales, cada veterinario tiene una sucursal asociada. Esta decisión es importante porque algunas queries necesitan vincular pacientes o atenciones con la sucursal del veterinario que las realizó.

Ejemplo de veterinario

```
1 {
2   "_id": "V001",
3   "nombre": "Gabriela",
```

```
4  "apellido": "Fontana",
5  "matricula": "VET-0041",
6  "especialidad": "Clinica General",
7  "sucursal": "Palermo",
8  "activo": true
9 }
```

Una de las decisiones más importantes del modelo fue guardar consultas y cirugías en una misma colección llamada `consultas`. Ambas representan atenciones clínicas y comparten la mayor parte de sus campos: paciente, veterinario, fecha, motivo, diagnóstico, costo y estado. Para diferenciarlas se utiliza el campo `tipo`. Esto evita duplicar pipelines y simplifica las queries que trabajan sobre historial clínico, diagnósticos frecuentes o ingresos por veterinario.

Ejemplo de consulta

```
1  {
2    "_id": "CON001",
3    "id_paciente": "P001",
4    "id_vet": "V001",
5    "fecha": "2025-10-05T00:00:00.000Z",
6    "tipo": "Consulta",
7    "motivo": "Control anual",
8    "diagnostico": "Sano",
9    "costo": 4500,
10   "estado": "Cerrada"
11 }
```

Las vacunaciones se guardan en una colección separada. Aunque también se relacionan con pacientes y veterinarios, su estructura tiene otra semántica: lo central es la vacuna aplicada y la fecha de próxima dosis. Mezclarlas con consultas hubiera generado documentos con campos opcionales y no aportaba una ventaja clara para las queries pedidas. Los campos `fecha_aplicacion` y `proxima_dosis` se cargan como fechas.

Ejemplo de vacunación

```
1  {
2    "_id": "VAC001",
3    "id_paciente": "P001",
4    "id_vet": "V001",
5    "fecha_aplicacion": "2025-10-05T00:00:00.000Z",
```

```

6  "nombre_vacuna": "Antirrabica",
7  "proxima_dosis": "2026-10-05T00:00:00.000Z"
8  }

```

Por último, la colección `productos` guarda el catálogo farmacéutico. En MongoDB se conserva la información descriptiva y relativamente estable de cada producto, como nombre, categoría, precio unitario, vencimiento y proveedor. La cantidad disponible no se guarda en esta colección porque forma parte del stock operativo, que cambia con mayor frecuencia y se administra en Redis.

Ejemplo de producto

```

1  {
2    "_id": "PRD001",
3    "nombre": "Amoxicilina 250mg",
4    "categoria": "Antibiotico",
5    "precio_unit": 850,
6    "vencimiento": "2026-06-30T00:00:00.000Z",
7    "proveedor": "VetFarma SA"
8  }

```

Además de las colecciones, se definieron índices sobre campos usados frecuentemente en filtros, relaciones o rangos. El seed crea índices sobre `fecha` e `id_vet` en `consultas`, sobre `id_propietario` en `pacientes`, y sobre `proveedor` y `vencimiento` en `productos`. Estos índices acompañan las consultas implementadas y evitan depender únicamente de recorridos completos sobre las colecciones.

También se define una vista formal para la Query 11: `vista_ingresos_vet_mes`. Esta vista usa un pipeline de agregación sobre consultas, cruza con veterinarios y expone los ingresos del mes actual agrupados por profesional.

3.2 Modelo en Redis

Redis se usa con un modelo más acotado que MongoDB. No se intentó replicar las entidades clínicas ni duplicar el dominio completo, sino guardar únicamente estructuras donde Redis aporta una ventaja clara: unidades disponibles del stock farmacéutico y caché de consultas pesadas.

La estructura principal es `stock:unidades`, un *Sorted Set* donde cada producto tiene como valor su identificador y como score la cantidad disponible. Esta estructura se carga desde el campo `unidades` del CSV de stock farmacéutico.

Estructura de stock en Redis

```
1 stock:unidades
2
3 value    score
4 PRD001   120
5 PRD004   40
6 PRD006   25
7 PRD017   15
```

El uso de un *Sorted Set* responde directamente a la necesidad de ordenar y filtrar por cantidad disponible. Para la consulta de stock bajo, Redis permite buscar por rango sobre el score y obtener rápidamente los productos por debajo de un umbral.

Consulta de stock bajo

```
1 ZRANGEBYSCORE stock:unidades -inf 49
```

El descuento de stock es el caso más sensible porque involucra concurrencia sobre un contador compartido. Por eso, la lógica de decremento se ejecuta dentro de Redis mediante un script Lua atómico, que verifica existencia, valida cantidad suficiente y descuenta en una única operación indivisible.

Además del stock, Redis guarda resultados cacheados de algunas queries del dashboard. Estas claves siguen el prefijo `cache:` y contienen resultados serializados en JSON con un TTL corto. Esto permite acelerar consultas frecuentes sin convertir esos resultados en fuente de verdad.

Las claves de caché usadas por el sistema son:

- `cache:vets-consultas-60d`, con TTL de 300 segundos.
- `cache:top-diagnosticos`, con TTL de 300 segundos.
- `cache:ingresos-vet-mes`, con TTL de 60 segundos.

Estas claves representan información derivada. Si expiran o se invalidan, el resultado puede recalcularse desde MongoDB. Por eso tienen una semántica distinta a `stock:unidades`: el stock representa estado operativo del sistema, mientras que la caché representa resultados temporales.

La separación por prefijos también permite operar sobre la caché sin afectar el stock. Por ejemplo, el endpoint de limpieza de caché elimina claves `cache:*`, pero no toca `stock:unidades`. Esto es importante porque ambos datos viven en Redis, aunque no tienen la misma responsabilidad dentro del sistema.

3.3 Relación entre ambos motores

La relación entre MongoDB y Redis aparece principalmente en el manejo del stock farmacéutico. MongoDB guarda el catálogo de productos, mientras que Redis guarda las unidades disponibles. Esto significa que para mostrar productos con stock bajo el sistema combina información de ambos motores: Redis identifica rápidamente qué productos están por debajo del umbral y MongoDB aporta los datos descriptivos de esos productos.

Este diseño evita duplicar información descriptiva. Redis no necesita saber el nombre, proveedor o vencimiento de cada producto; solo necesita saber cuántas unidades quedan. MongoDB, por su parte, no necesita actualizar constantemente un contador de stock que cambia con cada consulta. Cada motor conserva la parte del dato que mejor se adapta a su modelo.

La relación también aparece en las queries cacheadas. En esos casos, MongoDB sigue siendo la fuente de verdad: la query se calcula sobre sus colecciones y el resultado se guarda temporalmente en Redis. Redis no reemplaza a MongoDB, sino que funciona como una capa de aceleración para lecturas frecuentes.

Esta separación tiene una limitación: no existe una transacción global entre ambos motores. Por ejemplo, al registrar una consulta que consume productos, el sistema debe insertar el documento de la consulta en MongoDB y descontar unidades en Redis. Para reducir el riesgo de inconsistencias, se valida el stock antes de insertar la consulta y se descuenta mediante una operación atómica en Redis. Sin embargo, la coordinación entre ambos motores sigue siendo una limitación inherente de la arquitectura elegida.

4 Implementación de las Queries

4.1 Queries resueltas con MongoDB

4.1.1 Query 1: Pacientes activos con datos de propietario

La primera query expone el endpoint `GET /api/pacientes-activos` y resuelve el listado de pacientes activos junto con los datos completos de su propietario. Se implementa sobre MongoDB utilizando las colecciones `pacientes` y `propietarios`.

La consulta primero filtra los documentos de `pacientes` cuyo campo `activo` vale `true`. Luego utiliza `$lookup` para unir cada paciente con su propietario, comparando `pacientes.id_propietario` contra `propietarios._id`. Como cada paciente tiene un único propietario, se aplica `$unwind` sobre el resultado de la unión y finalmente se proyectan los campos relevantes.

Esta query aprovecha directamente la decisión de modelar pacientes y propietarios como colecciones separadas vinculadas por referencia. De esta forma, no se duplica la información del propietario dentro de cada paciente, pero se puede reconstruir el resultado combinado cuando el caso de uso lo requiere.

La proyección incluye los datos del paciente activo y todos los campos del propietario cargados por el seed: identificador, nombre, apellido, DNI, email, teléfono, ciudad, provincia y estado lógico.

Ejemplo de resultado de Query 1

```
1  [
2    {
3      "_id": "P005",
4      "nombre": "Rex",
5      "especie": "Perro",
6      "raza": "Pastor Aleman",
7      "fecha_nac": "2020-09-08T00:00:00.000Z",
8      "activo": true,
9      "propietario": {
10       "_id": "C004",
11       "nombre": "Diego",
12       "apellido": "Herrera",
13       "dni": "35789012",
14       "email": "diego@gmail.com",
15       "telefono": "1178004567",
16       "ciudad": "Mendoza",
```

```

17     "provincia": "Mendoza",
18     "activo": true
19   }
20 }
21 ]

```

4.1.2 Query 2: Consultas en seguimiento con veterinario y costo

La segunda query expone el endpoint `GET /api/consultas-seguimiento` y lista las consultas médicas abiertas cuyo estado es `Seguimiento`. Se resuelve íntegramente en MongoDB usando las colecciones `consultas`, `veterinarios` y `pacientes`.

El pipeline comienza con un `$match` sobre `consultas.estado` para quedarse únicamente con las atenciones en seguimiento. Luego realiza un `$lookup` contra `veterinarios` usando `id_vet`, lo que permite mostrar el veterinario asignado a la atención. Después hace un segundo `$lookup` contra `pacientes` para incorporar datos básicos del animal atendido. En ambos casos se usa `$unwind`, porque cada consulta referencia a un único veterinario y a un único paciente.

La proyección final conserva los datos principales de la consulta, incluyendo fecha, tipo, motivo, diagnóstico, estado y costo. También incluye el identificador del paciente, un bloque con datos básicos del paciente y otro bloque con los datos profesionales del veterinario. El resultado se ordena por fecha descendente para mostrar primero las atenciones más recientes que todavía requieren seguimiento.

Esta query aprovecha el campo `estado` de la colección `consultas`, que permite distinguir atenciones cerradas de atenciones abiertas. También aprovecha que las atenciones clínicas, incluidas las cirugías, comparten una misma colección: el seguimiento operativo se puede consultar desde `consultas` sin duplicar lógica en otra entidad.

Ejemplo de resultado de Query 2

```

1  [
2    {
3      "_id": "CON004",
4      "id_paciente": "P004",
5      "fecha": "2025-11-01T00:00:00.000Z",
6      "tipo": "Consulta",
7      "motivo": "Perdida de apetito",
8      "diagnostico": "Estres ambiental",
9      "costo": 3800,
10     "estado": "Seguimiento",

```

```

11     "paciente": {
12         "_id": "P004",
13         "nombre": "Nube",
14         "especie": "Conejo",
15         "raza": "Angora"
16     },
17     "veterinario": {
18         "_id": "V001",
19         "nombre": "Gabriela",
20         "apellido": "Fontana",
21         "matricula": "VET-0041",
22         "especialidad": "Clinica General",
23         "sucursal": "Palermo"
24     }
25 }
26 ]

```

4.1.3 Query 3: Historial completo de un paciente

La tercera query expone el endpoint `GET /api/historial-paciente/:id` y reconstruye el historial clínico de un paciente a partir de sus consultas y vacunaciones. Se resuelve principalmente con MongoDB, usando las colecciones `pacientes`, `consultas`, `vacunaciones` y `veterinarios`, y luego combina los resultados en Node.js.

Antes de ejecutar los pipelines, la función valida que el paciente solicitado exista. Si el identificador no corresponde a ningún documento de `pacientes`, se devuelve un error. Esta validación evita construir historiales sobre pacientes inexistentes.

La implementación ejecuta dos agregaciones en paralelo. La primera consulta la colección `consultas`, filtra por `id_paciente`, une cada atención con su veterinario mediante `$lookup` y proyecta el evento con un formato común. La segunda hace lo mismo sobre `vacunaciones`, usando `fecha_aplicacion` como fecha del evento e incorporando los campos propios de vacunas, como `nombre_vacuna` y `proxima_dosis`.

Se decidió realizar la combinación final en la capa de servicio porque el historial integra eventos de dos colecciones con estructuras distintas. MongoDB concentra el trabajo más pesado de cada origen: filtra por paciente, resuelve la unión con veterinarios y proyecta cada evento con los campos necesarios. Node.js queda limitado a una tarea de orquestación: llevar ambas listas a un formato común, unirlas y aplicar el criterio de ordenamiento final.

Ambos resultados se normalizan a una misma estructura con campos como `tipo_evento`, `id_evento`, `fecha`, `paciente` y `veterinario`. Luego se concatenan en Node.js y se or-

denan por fecha descendente, mostrando primero los eventos más recientes. En caso de empate de fecha, se usa el identificador del evento como segundo criterio de orden.

Esta query justifica dos decisiones de modelado. Por un lado, consultas y cirugías comparten la colección `consultas`, por lo que todas las atenciones clínicas del paciente se recuperan desde el mismo origen. Por otro lado, vacunaciones se mantiene como colección separada porque tiene una semántica distinta, pero puede integrarse al historial mediante una normalización final del resultado.

Fragmento de resultado de Query 3 para P001

```
1  [
2    {
3      "tipo_evento": "Consulta",
4      "id_evento": "CON007",
5      "fecha": "2025-12-02T00:00:00.000Z",
6      "paciente": {
7        "_id": "P001",
8        "nombre": "Firulais",
9        "especie": "Perro",
10       "raza": "Labrador"
11     },
12     "veterinario": {
13       "_id": "V001",
14       "nombre": "Gabriela",
15       "apellido": "Fontana",
16       "matricula": "VET-0041",
17       "sucursal": "Palermo"
18     },
19     "motivo": "Control post-vacuna",
20     "diagnostico": "Sin novedades",
21     "costo": 2000,
22     "estado": "Cerrada",
23     "nombre_vacuna": null,
24     "proxima_dosis": null
25   },
26   {
27     "tipo_evento": "Vacunacion",
28     "id_evento": "VAC001",
29     "fecha": "2025-10-05T00:00:00.000Z",
30     "paciente": {
31       "_id": "P001",
```

```

32     "nombre": "Firulais",
33     "especie": "Perro",
34     "raza": "Labrador"
35 },
36 "veterinario": {
37     "_id": "V001",
38     "nombre": "Gabriela",
39     "apellido": "Fontana",
40     "matricula": "VET-0041",
41     "sucursal": "Palermo"
42 },
43 "motivo": null,
44 "diagnostico": null,
45 "costo": null,
46 "estado": null,
47 "nombre_vacuna": "Antirrabica",
48 "proxima_dosis": "2026-10-05T00:00:00.000Z"
49 }
50 ]

```

4.1.4 Query 4: Propietarios con más de un paciente

La cuarta query expone el endpoint `GET /api/proprietarios-multiples-pacientes` y lista los propietarios que tienen más de un paciente registrado. Se implementa sobre MongoDB usando las colecciones `pacientes` y `propietarios`.

El pipeline comienza desde `pacientes`, porque el dato que se necesita contar es la cantidad de mascotas asociadas a cada propietario. Primero agrupa por `id_propietario`, calcula `cantidad_pacientes` con `$sum` y guarda en un arreglo los datos básicos de cada paciente mediante `$push`. Luego aplica un `$match` para conservar únicamente los grupos con más de un paciente.

Después de identificar los propietarios que cumplen la condición, la query usa `$lookup` contra `propietarios` para incorporar sus datos de contacto. Como cada grupo corresponde a un único propietario, se aplica `$unwind` y finalmente se proyecta una salida con el identificador del propietario, la cantidad total de pacientes, los datos del propietario y el detalle de las mascotas asociadas.

La consulta cuenta pacientes registrados, no solamente pacientes activos, porque la consigna no restringe este punto a mascotas activas. Por eso el resultado conserva el campo `activo` en cada paciente: permite ver si alguna de las mascotas asociadas está dada de baja lógicamente sin excluirla del conteo. El resultado se ordena por cantidad de

pacientes en forma descendente y, ante empate, por identificador de propietario.

Ejemplo de resultado de Query 4

```
1  [
2    {
3      "cantidad_pacientes": 3,
4      "pacientes": [
5        {
6          "_id": "P001",
7          "nombre": "Firulais",
8          "especie": "Perro",
9          "raza": "Labrador",
10         "activo": true
11       },
12       {
13         "_id": "P003",
14         "nombre": "Coco",
15         "especie": "Perro",
16         "raza": "Beagle",
17         "activo": false
18       },
19       {
20         "_id": "P018",
21         "nombre": "Pelusa",
22         "especie": "Conejo",
23         "raza": "Holland Lop",
24         "activo": true
25       }
26     ],
27     "propietario": {
28       "_id": "C001",
29       "nombre": "Ana",
30       "apellido": "Rodriguez",
31       "email": "ana@gmail.com",
32       "telefono": "1145001234",
33       "ciudad": "Buenos Aires",
34       "provincia": "Buenos Aires",
35       "activo": true
36     },
37     "id_propietario": "C001"
38   }
```

4.1.5 Query 6: Pacientes con vacunas vencidas

La sexta query expone el endpoint `GET /api/pacientes-vacunas-vencidas` y lista los pacientes que tienen al menos una vacunación cuya próxima dosis es anterior a la fecha actual. Se resuelve íntegramente en MongoDB usando las colecciones `vacunaciones` y `pacientes`.

El pipeline parte de `vacunaciones`, porque el criterio de vencimiento está en el campo `proxima_dosis`. La función toma la fecha del sistema, la normaliza al inicio del día y filtra con `$match` las vacunaciones donde `proxima_dosis < hoy`. Esta normalización evita marcar como vencida una vacuna cuya próxima dosis sea el mismo día de ejecución.

Después, la query agrupa por paciente. Para cada uno calcula la cantidad de vacunas vencidas, conserva el detalle en un arreglo y obtiene la próxima dosis más antigua con `$min`. Luego realiza un `$lookup` contra `pacientes` para incorporar los datos básicos del animal y proyecta el resultado final.

El ordenamiento usa primero `proxima_dosis_mas_antigua` en forma ascendente, de modo que los pacientes con vencimientos más antiguos aparezcan antes. Esto permite usar la consulta como una lista de priorización operativa para contactar propietarios o programar revacunaciones.

Esta query justifica mantener las vacunaciones como colección separada de las consultas. El vencimiento de una vacuna no depende del diagnóstico ni del costo de una atención, sino de una fecha propia del calendario sanitario del paciente.

Ejemplo de resultado de Query 6

```

1  [
2    {
3      "cantidad_vacunas_vencidas": 2,
4      "vacunas_vencidas": [
5        {
6          "id_vacuna": "VAC022",
7          "nombre_vacuna": "Sextuple",
8          "fecha_aplicacion": "2024-06-10T00:00:00.000Z",
9          "proxima_dosis": "2025-06-10T00:00:00.000Z",
10         "id_vet": "V001"
11       },
12       {
13         "id_vacuna": "VAC031",

```

```

14     "nombre_vacuna": "Triple Felina",
15     "fecha_aplicacion": "2024-04-15T00:00:00.000Z",
16     "proxima_dosis": "2025-04-15T00:00:00.000Z",
17     "id_vet": "V001"
18   }
19 ],
20 "proxima_dosis_mas_antigua": "2025-04-15T00:00:00.000Z",
21 "paciente": {
22   "_id": "P003",
23   "nombre": "Coco",
24   "especie": "Perro",
25   "raza": "Beagle",
26   "activo": false
27 }
28 }
29 ]

```

4.1.6 Query 9: Consultas de control con costo menor a \$5.000

La novena query expone el endpoint `GET /api/controles-baratos?max=5000` y lista las consultas de control cuyo costo es menor al umbral indicado. Se resuelve con MongoDB usando las colecciones `consultas` y `pacientes`.

En los datos cargados, el campo `tipo` distingue principalmente entre `Consulta` y `Cirugia`; no existe un tipo separado llamado `Control`. Por eso se interpreta “control” a partir del campo `motivo`, seleccionando las consultas cuyo motivo comienza con esa palabra. Al mismo tiempo, se mantiene el filtro `tipo = Consulta` para excluir cirugías y otras atenciones clínicas que no correspondan al caso pedido.

El pipeline aplica un `$match` compuesto sobre `tipo`, `motivo` y `costo`. Luego hace un `$lookup` contra `pacientes` para mostrar el animal atendido, aplica `$unwind` porque cada consulta tiene un único paciente, ordena por costo ascendente y proyecta los datos principales de la atención.

Ejemplo de resultado de Query 9

```

1  [
2    {
3      "_id": "CON007",
4      "fecha": "2025-12-02T00:00:00.000Z",
5      "motivo": "Control post-vacuna",
6      "diagnostico": "Sin novedades",

```

```

7     "costo": 2000,
8     "paciente": {
9         "nombre": "Firulais",
10        "especie": "Perro"
11    }
12 }
13 ]

```

4.1.7 Query 10: Pacientes de una sucursal determinada

La décima query expone el endpoint de pacientes por sucursal y lista los pacientes asociados a una sucursal determinada. Se resuelve con MongoDB usando las colecciones de `veterinarios`, `consultas`, `vacunaciones` y `pacientes`.

La sucursal no está guardada en el documento del paciente, sino en el veterinario. Por eso el pipeline comienza desde `veterinarios` y filtra por el nombre de sucursal recibido como parámetro. A partir de esos veterinarios, la query busca pacientes atendidos tanto en `consultas` como en `vacunaciones`.

Después de traer ambos conjuntos, se usa `$setUnion` para unir los identificadores de pacientes sin duplicados. Luego se desarma el arreglo, se agrupa nuevamente por paciente para deduplicar entre distintos veterinarios de la misma sucursal y se hace un `$lookup` final contra `pacientes` para devolver el documento completo del animal.

Fragmento de resultado de Query 10 para Palermo

```

1  [
2    {
3      "_id": "P001",
4      "nombre": "Firulais",
5      "especie": "Perro",
6      "raza": "Labrador",
7      "fecha_nac": "2019-04-12T00:00:00.000Z",
8      "id_propietario": "C001",
9      "activo": true
10   },
11   {
12     "_id": "P002",
13     "nombre": "Michi",
14     "especie": "Gato",
15     "raza": "Siames",
16     "fecha_nac": "2021-07-03T00:00:00.000Z",

```

```

17     "id_propietario": "C002",
18     "activo": true
19   }
20 ]

```

4.1.8 Query 12: Propietarios a revisar

La duodécima query expone el endpoint `GET /api/proprietarios-inactivos`. Para esta consulta, consideramos que el listado de propietarios inactivos como aquellos propietarios dados de baja, propietarios sin mascotas registradas y propietarios activos con mascotas pero sin consultas recientes.

La función calcula el límite temporal a partir de la fecha del sistema, tomando el inicio del día ubicado un año atrás. Luego parte de `propietarios`, une cada dueño con sus pacientes mediante `$lookup` y, con esos identificadores de pacientes, busca consultas cuya fecha sea mayor o igual a ese límite.

Después agrega la cantidad de mascotas y la cantidad de consultas recientes. La condición final conserva propietarios que cumplan al menos una de las tres condiciones definidas. Esta interpretación permite detectar dueños sin actividad vigente desde distintas causas: baja administrativa, ausencia de mascotas o falta de consultas durante el período analizado.

Para que el resultado sea fácil de leer, la query asigna una `categoria` con `$switch`. La categoría `Dado de baja` tiene prioridad sobre las demás; luego aparece `Sin mascotas`; y, como caso restante, `Sin consultas en el último año`. Ese mismo criterio se usa para ordenar el resultado. La consulta considera consultas, no vacunaciones, porque el enunciado pide específicamente consultas registradas.

Fragmento de resultado de Query 12

```

1  [
2    {
3      "_id": "C010",
4      "nombre": "Martin",
5      "apellido": "Acosta",
6      "email": "martin@gmail.com",
7      "ciudad": "Bahia Blanca",
8      "provincia": "Buenos Aires",
9      "activo": false,
10     "cantidad_mascotas": 1,
11     "cantidad_consultas_recientes": 1,

```

```

12     "categoria": "Dado de baja"
13 },
14 {
15     "_id": "C009",
16     "nombre": "Sofia",
17     "apellido": "Mendez",
18     "email": "sofi@gmail.com",
19     "ciudad": "San Miguel de Tucuman",
20     "provincia": "Tucuman",
21     "activo": true,
22     "cantidad_mascotas": 0,
23     "cantidad_consultas_recientes": 0,
24     "categoria": "Sin mascotas"
25 },
26 {
27     "_id": "C008",
28     "nombre": "Federico",
29     "apellido": "Russo",
30     "email": "fedegmail.com",
31     "ciudad": "Mar del Plata",
32     "provincia": "Buenos Aires",
33     "activo": true,
34     "cantidad_mascotas": 1,
35     "cantidad_consultas_recientes": 0,
36     "categoria": "Sin consultas en el ultimo anio"
37 }
38 ]

```

4.1.9 Query 13: ABM de propietarios

La decimotercera query implementa el ABM completo de propietarios. Se resuelve sobre MongoDB usando la colección `propietarios` y expone tres endpoints:

- `POST /api/propietarios`
- `PUT /api/propietarios/:id`
- `DELETE /api/propietarios/:id`

El alta valida que el documento recibido tenga `_id` y que no exista otro propietario con ese mismo identificador. Si la validación pasa, inserta el documento recibido en

`propietarios`. El campo `activo` queda con valor `true` por defecto si no viene informado, pero se respeta el valor enviado en el alta.

La modificación toma el identificador desde la URL y aplica un `$set` únicamente con los campos enviados por el cliente. Antes de actualizar, se elimina `_id` del payload para impedir que una modificación cambie la clave primaria del documento. Si no existe un propietario con ese identificador, la función devuelve un error.

La baja se implementa como baja lógica. En lugar de borrar físicamente el documento, actualiza `activo` a `false`. Esto permite conservar trazabilidad y evita dejar referencias huérfanas desde pacientes, consultas o historiales que ya estaban asociados a ese propietario.

Ejemplos de respuesta de Query 13

```
1 {
2   "alta": {
3     "ok": true,
4     "_id": "C999"
5   },
6   "modificacion": {
7     "ok": true,
8     "modificados": 1
9   },
10  "baja_logica": {
11    "ok": true,
12    "baja_logica": "C999"
13  }
14 }
```

4.2 Queries resueltas con Redis

4.2.1 Query 15: Decremento de stock tras una consulta

La decimoquinta query expone el endpoint `POST /api/decrementar-stock` y descuenta unidades de un producto del stock farmacéutico. Se resuelve íntegramente en Redis usando la clave `stock:unidades`.

El stock está modelado como un *Sorted Set*: cada producto es un miembro del conjunto y la cantidad disponible se guarda como score. La función recibe `id_producto` y `cantidad`; antes de ejecutar el descuento valida que el producto esté informado y que la cantidad sea un entero positivo.

El decremento se realiza mediante un script Lua ejecutado con `redis.eval`. Dentro

de ese script Redis verifica que el producto exista, valida que el stock actual alcance para cubrir la cantidad solicitada y descuenta las unidades con `ZINCRBY`. Como el script se ejecuta de forma atómica, dos requests concurrentes no pueden validar ambos contra el mismo stock inicial y dejar unidades negativas.

El script devuelve códigos simples: `-1` si el producto no existe, `-2` si el stock es insuficiente, y el nuevo valor de unidades cuando el descuento se realiza correctamente. La función traduce esos códigos a errores de negocio o a una respuesta con las unidades restantes.

Ejemplo de resultado de Query 15

```
1 {  
2   "id_producto": "PRD004",  
3   "unidades": 38  
4 }
```

4.3 Queries que combinan MongoDB y Redis

4.3.1 Query 8: Stock de productos con menos de 50 unidades

La octava query expone el endpoint `GET /api/stock-bajo?umbral=50` y lista productos cuyo stock se encuentra por debajo del umbral indicado. Es una query polígota: Redis identifica rápidamente los productos con pocas unidades y MongoDB aporta la información descriptiva del catálogo.

En Redis se usa la clave `stock:unidades`, modelada como un *Sorted Set*. El identificador del producto es el valor del conjunto y la cantidad disponible es el score. La consulta usa un rango por score para obtener los productos con unidades menores al umbral, manteniendo el orden ascendente por stock.

Luego, con los identificadores devueltos por Redis, la función consulta la colección `productos` en MongoDB mediante `$in`. MongoDB devuelve los campos de catálogo, como nombre, categoría, precio unitario, vencimiento y proveedor. Finalmente, Node.js combina ambos resultados conservando el orden que vino de Redis.

Esta query justifica la separación entre catálogo y stock operativo. MongoDB no necesita actualizar un contador de unidades con cada operación, y Redis no necesita duplicar los datos descriptivos del producto. Cada motor conserva la parte del dato que resuelve mejor.

Fragmento de resultado de Query 8

```
1  [  
2    {  
3      "_id": "PRD017",  
4      "nombre": "Insulina canina",  
5      "categoria": "Hormonal",  
6      "precio_unit": 8500,  
7      "vencimiento": "2026-08-15T00:00:00.000Z",  
8      "proveedor": "MediAnimal",  
9      "unidades": 15  
10   },  
11   {  
12     "_id": "PRD015",  
13     "nombre": "Praziquantel 50mg",  
14     "categoria": "Antiparasitario",  
15     "precio_unit": 1300,  
16     "vencimiento": "2026-06-30T00:00:00.000Z",  
17     "proveedor": "VetFarma SA",  
18     "unidades": 20  
19   }  
20  ]
```

4.3.2 Query 14: Registro de nueva consulta médica

La decimocuarta query expone el endpoint `POST /api/consultas` y registra una nueva atención médica. Se apoya principalmente en MongoDB, usando las colecciones `pacientes`, `veterinarios` y `consultas`, pero también puede interactuar con Redis si la consulta informa productos consumidos.

Antes de insertar la consulta, la función valida que el paciente exista y esté activo. También valida que el veterinario exista y esté activo. Estas validaciones son necesarias porque `consultas` guarda referencias por identificador; insertar una atención con un paciente o veterinario inexistente rompería la integridad referencial del modelo.

Si el request incluye `productos_usados`, la función consolida productos repetidos y valida que cada cantidad sea un entero positivo. Luego consulta Redis para comprobar que cada producto exista en `stock:unidades` y que haya unidades suficientes. La consolidación evita validar dos líneas del mismo producto como si fueran consumos independientes contra el mismo stock inicial.

Para el documento de la consulta, si el cliente no envía `_id`, la función genera el siguiente identificador con prefijo `CON`. Luego inserta la atención en MongoDB con

fecha, tipo, motivo, diagnóstico, costo y estado. Una vez registrada la consulta, invalida las claves de caché que dependen de la colección `consultas`: `cache:top-diagnostics`, `cache:ingresos-vet-mes` y `cache:vets-consultas-60d`.

Cuando hay productos usados, el descuento se delega en la Query 15. De esta forma, la validación de stock y el decremento final usan la misma estructura de Redis y el descuento efectivo se realiza con el script Lua atómico.

Ejemplo de resultado de Query 14

```
1  {
2    "ok": true,
3    "consulta": {
4      "_id": "CON999",
5      "id_paciente": "P001",
6      "id_vet": "V001",
7      "fecha": "2026-06-06T00:00:00.000Z",
8      "tipo": "Consulta",
9      "motivo": "Control de rutina",
10     "diagnostico": "Sano",
11     "costo": 4500,
12     "estado": "Cerrada"
13   },
14   "stock": [
15     {
16       "id_producto": "PRD001",
17       "unidades": 118
18     }
19   ]
20 }
```

4.4 Queries cacheadas

Las queries cacheadas usan un patrón *cache-aside*. La aplicación primero consulta Redis con una clave determinada. Si el valor existe, lo deserializa desde JSON y devuelve ese resultado. Si no existe, ejecuta la agregación correspondiente en MongoDB, serializa el resultado, lo guarda en Redis con un TTL y finalmente lo devuelve al cliente.

Este patrón se eligió porque MongoDB sigue siendo la fuente de verdad para los datos clínicos y administrativos. Redis solamente acelera lecturas frecuentes del dashboard. En consecuencia, los resultados cacheados son descartables: si una clave expira o se borra, la query vuelve a calcularse contra MongoDB.

Para hacer visible este comportamiento, el servidor agrega el header `X-Cache` en las respuestas cacheadas. El valor puede indicar `MISS` cuando se recalcula el resultado o `HIT` cuando se responde desde Redis. La implementación usa `AsyncLocalStorage` para asociar esos eventos de caché al request HTTP actual, evitando usar variables globales compartidas entre requests concurrentes.

4.4.1 Query 5: Veterinarios activos con consultas en los últimos 60 días

La quinta query expone el endpoint `GET /api/vets-activos-consultas-60d` y lista los veterinarios activos junto con la cantidad de consultas registradas en los últimos 60 días. Se calcula con MongoDB sobre las colecciones `veterinarios` y `consultas`, y el resultado se guarda temporalmente en Redis como caché.

La función toma la fecha del sistema y construye un rango de días calendario: desde el inicio del día ubicado 60 días atrás hasta el inicio del día siguiente al de ejecución. Este criterio es importante porque los datos del proyecto se cargan como fechas de día completo, no como instantes con hora clínica.

El pipeline comienza desde `veterinarios` con un `$match` sobre `activo: true`. Luego usa un `$lookup` con pipeline correlacionado contra `consultas`: compara el veterinario de cada atención con el documento activo que se está procesando y filtra la fecha dentro del rango calculado. En la proyección final, la cantidad se obtiene con `$size` sobre el arreglo de consultas recientes.

El resultado incluye también veterinarios activos sin consultas recientes, porque la consigna pide listar veterinarios activos y su cantidad de consultas. Finalmente se ordena por `cantidad_consultas_60d` en forma descendente y, ante empates, por apellido y nombre.

Esta query es buena candidata para caché porque cruza veterinarios con consultas y puede ejecutarse repetidamente desde el dashboard. Por eso se guarda en Redis bajo la clave `cache:vets-consultas-60d` con un TTL de 300 segundos. Cuando se registra una nueva consulta médica, la función de alta invalida esta clave para que el próximo pedido recalcule el resultado desde MongoDB.

Ejemplo de resultado de Query 5

```
1  [  
2    {  
3      "nombre": "Gabriela",  
4      "apellido": "Fontana",  
5      "matricula": "VET-0041",  
6      "especialidad": "Clinica General",  
7      "sucursal": "Palermo",
```

```

8     "activo": true,
9     "id_vet": "V001",
10    "cantidad_consultas_60d": 8
11  }
12 ]

```

4.4.2 Query 7: Top 5 diagnósticos más frecuentes

La séptima query expone el endpoint `GET /api/top-diagnostics` y obtiene los cinco diagnósticos más frecuentes registrados en las atenciones. Se calcula con MongoDB sobre la colección `consultas` y se guarda temporalmente en Redis como caché.

El pipeline agrupa los documentos por `diagnostico`, cuenta ocurrencias con `$sum`, ordena de mayor a menor por cantidad y limita el resultado a cinco elementos. La salida conserva el diagnóstico como identificador del grupo y agrega el campo `total`.

Esta query aprovecha que consultas y cirugías se guardan en la misma colección de atenciones clínicas. De esa forma, el análisis de diagnósticos frecuentes se calcula sobre el historial clínico completo que vive en `consultas`, sin tener que duplicar el pipeline para otra colección.

Como es una agregación que puede consultarse muchas veces desde el dashboard y no necesita recalcularse en cada lectura, el resultado se cachea en Redis bajo la clave `cache:top-diagnostics` con un TTL de 300 segundos. Cuando se registra una nueva consulta médica, la función de alta invalida esta clave para que el siguiente pedido vuelva a calcular el ranking desde MongoDB.

Ejemplo de resultado de Query 7

```

1  [
2    {
3      "_id": "Sano",
4      "total": 7
5    },
6    {
7      "_id": "Otitis externa",
8      "total": 5
9    },
10   {
11     "_id": "Dermatitis atopica",
12     "total": 4
13   },

```

```

14   {
15     "_id": "Infestacion parasitaria",
16     "total": 3
17   },
18   {
19     "_id": "Cirugia exitosa",
20     "total": 2
21   }
22 ]

```

4.4.3 Query 11: Ingresos por veterinario en el mes actual

La undécima query expone el endpoint `GET /api/ingresos-vet-mes`. Su resultado se obtiene leyendo una vista formal de MongoDB.

La vista se llama `vista_ingresos_vet_mes`. Se define sobre la colección `consultas` y usa un pipeline de agregación para resumir las atenciones por veterinario.

El pipeline de la vista calcula el rango del mes actual dentro de MongoDB usando la variable `$$NOW`. A partir de ese valor, filtra las consultas cuya fecha pertenece al mes en curso, agrupa por veterinario, suma el campo `costo` como ingresos y cuenta cuántas atenciones componen ese total.

Luego, la vista cruza el resultado con `veterinarios` para incorporar el nombre completo del profesional y su sucursal. La salida queda compuesta por el identificador del veterinario, el nombre del veterinario, la sucursal, el total de ingresos y la cantidad de atenciones. El resultado se ordena por ingresos descendentes.

Esta vista aprovecha que consultas y cirugías comparten la colección `consultas`: ambas representan atenciones clínicas con costo y participan del cálculo de ingresos. Para que la consulta tenga datos representativos independientemente del mes en que se ejecute el seed, se agregan algunas consultas con fechas relativas al mes actual. Además, el endpoint guarda en Redis durante 60 segundos el resultado de leer la vista, porque las vistas estándar de MongoDB se calculan al consultarse y no almacenan el resultado materializado.

Ejemplo de resultado de Query 11

```

1  [
2    {
3      "ingresos": 25000,
4      "cantidad": 1,
5      "id_vet": "V006",

```

```
6     "veterinario": "Mateo Bianchi",  
7     "sucursal": "Recoleta"  
8 }  
9 ]
```


5 Manual de Usuario

Esta sección explica cómo levantar y usar el sistema desde cero. La idea es que alguien que clone el repositorio pueda cargar las bases, abrir la interfaz, ejecutar las consultas y probar las operaciones de escritura sin tener que leer el código primero.

5.1 Requisitos previos

Para correr el proyecto localmente se necesita tener instalados Docker, Docker Compose, Node.js y npm. En nuestro caso usamos Docker para levantar MongoDB y Redis, y Node.js para ejecutar el seed y la API Express.

Antes de empezar conviene revisar que estén libres los siguientes puertos:

- `27017`, usado por MongoDB.
- `6379`, usado por Redis.
- `3000`, usado por la API y el dashboard web.

Si alguno de esos puertos ya está ocupado, el contenedor o el servidor pueden fallar al iniciar. En ese caso hay que detener el proceso que lo está usando o cambiar la configuración correspondiente.

5.2 Primera ejecución

Los comandos se ejecutan desde la carpeta raíz del proyecto. En este caso, esa carpeta es `vetsalud-bdd`. El orden recomendado es el siguiente:

Levantar el proyecto

```
1 docker compose up -d
2 cp .env.example .env
3 npm install
4 npm run seed
5 npm start
```

El primer comando levanta los contenedores de MongoDB y Redis. El archivo `.env` define las URLs de conexión y el nombre de la base de datos. El comando `npm install` instala las dependencias de Node.js, `npm run seed` carga los datos iniciales y `npm start` deja la API escuchando en `http://localhost:3000`.

Cuando el servidor queda iniciado, en la terminal debería aparecer un mensaje similar a:

Servidor iniciado

```
1 API escuchando en http://localhost:3000
```

5.3 Carga inicial de datos

El seed toma los archivos CSV de la carpeta `data` y carga la información en MongoDB y Redis. En MongoDB se cargan propietarios, pacientes, veterinarios, consultas, vacunaciones y productos. En Redis se carga el stock de productos dentro de la clave `stock:unidades`.

También se crean índices en MongoDB y se recrea la vista de ingresos usada por la Query 11. Además, el seed agrega algunas consultas con fechas relativas al mes actual para que la vista de ingresos mensuales tenga datos aunque el proyecto se ejecute en otro momento.

Es importante tener presente que `npm run seed` reinicia los datos de trabajo. Esto es útil para volver a un estado conocido después de probar altas, bajas o descuentos de stock. Por ejemplo, si durante una demo se descuentan unidades o se agrega un propietario de prueba, al correr nuevamente el seed se vuelve al estado inicial del dataset.

5.4 Uso del dashboard

El sistema incluye una interfaz web simple en `http://localhost:3000`. Sirve para probar de forma cómoda las 15 consultas y servicios pedidos.

La pantalla tiene dos pestañas principales:

- **Dashboard:** agrupa las acciones por tema. Hay bloques para pacientes y propietarios, atenciones, reportes y stock.
- **Queries:** muestra las consultas de forma más directa, con el endpoint que se está ejecutando y, cuando corresponde, los parámetros o el cuerpo JSON.

Cada vez que se ejecuta una acción, el resultado aparece en el panel inferior. Por defecto se muestra como tabla cuando el formato lo permite. Si se quiere ver la respuesta cruda, se puede usar el botón `Ver JSON`. Esto ayuda bastante cuando una respuesta tiene objetos anidados, por ejemplo propietarios con pacientes, historiales clínicos o consultas con productos usados.

En las queries cacheadas, la interfaz muestra una etiqueta de caché cuando la respuesta incluye el header `X-Cache`. Si aparece `MISS`, significa que el resultado se calculó y luego se guardó en Redis. Si aparece `HIT`, significa que se devolvió desde la caché. Las queries donde esto se ve especialmente son Q5, Q7 y Q11.

5.5 Consultas disponibles desde la interfaz

Las consultas de lectura se pueden ejecutar desde el dashboard sin escribir comandos. Algunas usan valores fijos por defecto para que sea fácil probarlas apenas se carga el seed.

- **Q1: Pacientes activos.** Lista pacientes activos con los datos de su propietario.
- **Q2: Consultas en seguimiento.** Muestra atenciones cuyo estado es `Seguimiento`.
- **Q3: Historial de paciente.** Requiere un id de paciente, por ejemplo `P001`.
- **Q4: Propietarios con múltiples pacientes.** Lista dueños con más de una mascota registrada.
- **Q5: Veterinarios activos con consultas en 60 días.** Devuelve la cantidad de consultas recientes por veterinario activo.
- **Q6: Vacunas vencidas.** Lista pacientes cuya próxima dosis es anterior al día actual.
- **Q7: Top diagnósticos.** Muestra los cinco diagnósticos más frecuentes.
- **Q8: Stock bajo.** Permite indicar un umbral; por defecto se usa `50`.
- **Q9: Controles económicos.** Permite indicar un costo máximo; por defecto se usa `5000`.
- **Q10: Pacientes por sucursal.** Permite elegir una sucursal, por ejemplo `Palermo`.
- **Q11: Ingresos por veterinario.** Lee la vista agregada del mes actual.
- **Q12: Propietarios a revisar.** Clasifica propietarios dados de baja, sin mascotas o sin consultas recientes.

5.6 ABM de propietarios

La Query 13 implementa alta, modificación y baja lógica de propietarios. Desde el dashboard se encuentra en la sección de pacientes y propietarios, con tres subpestañas: *Alta*, *Modificar* y *Baja*.

Para dar de alta un propietario se debe indicar al menos un `_id`. El sistema valida que no exista otro propietario con el mismo identificador. Si no se informa `activo`, se toma `true` como valor por defecto; si se envía el campo, se respeta el valor recibido.

Un ejemplo de alta desde la API sería:

Alta de propietario

```
1 curl -X POST http://localhost:3000/api/propietarios \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "_id": "C100",  
5     "nombre": "Lucia",  
6     "apellido": "Vega",  
7     "dni": "40111222",  
8     "email": "lu@mail.com",  
9     "telefono": "1144222333",  
10    "ciudad": "CABA",  
11    "provincia": "Buenos Aires"  
12  }'
```

Para modificar un propietario se usa el identificador en la URL y se envían solamente los campos que se quieren cambiar. El sistema ignora un eventual `_id` dentro del body para evitar que se cambie la clave primaria.

Modificación de propietario

```
1 curl -X PUT http://localhost:3000/api/propietarios/C100 \  
2   -H "Content-Type: application/json" \  
3   -d '{ "ciudad": "Rosario" }'
```

La baja se hace de forma lógica. Esto conserva el historial y evita perder referencias desde pacientes o consultas.

Baja lógica de propietario

```
1 curl -X DELETE http://localhost:3000/api/propietarios/C100
```

5.7 Registro de consultas

La Query 14 registra una nueva consulta médica. Desde el dashboard se encuentra en el bloque *Registrar consulta*. Para cargar una atención se debe indicar un paciente y un veterinario existentes. Además, ambos deben estar activos.

Los campos principales son:

- `id_paciente`: identificador del paciente atendido.
- `id_vet`: identificador del veterinario que atiende.

- `tipo`: puede ser `Consulta` o `Cirugia`.
- `motivo`: motivo de la atención.
- `diagnostico`: diagnóstico registrado.
- `costo`: importe de la atención.
- `estado`: por ejemplo `Cerrada` o `Seguimiento`.
- `productos_usados`: lista opcional de productos consumidos.

Si se cargan productos usados, el sistema valida el stock disponible en Redis. La cantidad debe ser positiva y entera. Si el stock alcanza, la consulta se inserta en MongoDB y luego se descuenta el stock usando la lógica atómica de la Query 15.

Ejemplo de alta de consulta con consumo de stock:

Alta de consulta

```

1  curl -X POST http://localhost:3000/api/consultas \
2    -H "Content-Type: application/json" \
3    -d '{
4      "id_paciente": "P002",
5      "id_vet": "V003",
6      "tipo": "Consulta",
7      "motivo": "Control",
8      "diagnostico": "Sano",
9      "costo": 3000,
10     "estado": "Cerrada",
11     "productos_usados": [
12       { "id_producto": "PRD001", "cantidad": 2 }
13     ]
14   }'
```

Después de registrar una consulta se invalidan las cachés que dependen de atenciones: diagnósticos frecuentes, ingresos por veterinario del mes y consultas recientes por veterinario. Por eso, si se ejecuta alguna de esas queries después de cargar una consulta, el primer pedido vuelve a recalcular el resultado.

5.8 Manejo de stock

El stock operativo se administra en Redis. Para ver productos con pocas unidades se usa Q8, que consulta el *Sorted Set* `stock:unidades` y luego completa los datos descriptivos desde MongoDB.

Para descontar stock directamente se usa Q15. Desde el dashboard alcanza con indicar el id del producto y la cantidad. Desde la API se puede probar así:

Decremento de stock

```
1 curl -X POST http://localhost:3000/api/decrementar-stock \  
2 -H "Content-Type: application/json" \  
3 -d '{ "id_producto": "PRD001", "cantidad": 3 }'
```

Si el producto no existe o si no hay unidades suficientes, la API responde con un error. La operación de descuento se ejecuta con Lua dentro de Redis, de modo que la validación y el decremento ocurren en una única operación atómica.

5.9 Uso directo de la API

Todos los servicios también pueden usarse directamente por HTTP. Los endpoints principales son:

- GET /api/pacientes-activos
- GET /api/consultas-seguimiento
- GET /api/historial-paciente/:id
- GET /api/propietarios-multiples-pacientes
- GET /api/vets-activos-consultas-60d
- GET /api/pacientes-vacunas-vencidas
- GET /api/top-diagnostics
- GET /api/stock-bajo?umbral=50
- GET /api/controles-baratos?max=5000
- GET /api/pacientes-sucursal?sucursal=Palermo
- GET /api/ingresos-vet-mes
- GET /api/propietarios-inactivos
- POST /api/propietarios
- PUT /api/propietarios/:id
- DELETE /api/propietarios/:id

- `POST /api/consultas`
- `POST /api/decrementar-stock`

La API devuelve JSON tanto en respuestas correctas como en errores. Cuando hay un error de validación, la respuesta tiene la forma:

Formato de error

```
1 {  
2   "error": "Mensaje descriptivo del problema"  
3 }
```

5.10 Caché

Redis también se usa como caché de algunas consultas de lectura. Esto se puede ver especialmente en:

- Q5, veterinarios activos con consultas en los últimos 60 días.
- Q7, top 5 diagnósticos.
- Q11, ingresos por veterinario en el mes actual.

Si se quiere limpiar manualmente la caché durante una prueba, se puede usar:

Limpiar caché

```
1 curl -X POST http://localhost:3000/api/cache/flush
```

Esto elimina las claves con prefijo `cache:`. No borra el stock ni los datos principales de MongoDB.

6 Resultados

El resultado final es una aplicación reproducible que permite levantar MongoDB y Redis con Docker, cargar los datos desde CSV y ejecutar las 15 consultas y servicios pedidos desde una API REST o desde el dashboard web.

La estructura de código quedó organizada por responsabilidad. Las consultas se separaron en módulos individuales dentro de `src/queries`, de modo que cada query tiene su propia función documentada. El servidor Express se limita a exponer los endpoints, validar parámetros simples y devolver las respuestas en JSON.

En MongoDB se cargan las colecciones principales del dominio: propietarios, pacientes, veterinarios, consultas, vacunaciones y productos. También se crean índices para acompañar filtros y relaciones frecuentes, y se define la vista de ingresos mensuales usada por la Query 11.

En Redis se carga el *Sorted Set* `stock:unidades` para el stock operativo. Además, se usan claves con prefijo `cache:` para guardar resultados derivados de forma temporal.

El dashboard permite probar las consultas de lectura, las operaciones de ABM, el alta de consultas y el decremento de stock. Además, muestra cuándo una query cacheada se resolvió desde Redis o cuándo fue recalculada contra MongoDB. Esto ayuda a verificar de forma práctica la separación entre fuente de verdad y caché.

También se validaron casos de escritura relevantes para el dominio: el alta de propietarios no permite duplicar identificadores, la baja se realiza de forma lógica, el alta de consultas valida paciente y veterinario existentes y activos, y el decremento de stock no permite dejar unidades negativas. En las operaciones de stock, Redis ejecuta la validación y el descuento mediante Lua para mantener la atomicidad del contador.

7 Limitaciones

La principal limitación técnica es que MongoDB y Redis no comparten una transacción global. El caso más sensible aparece al registrar una consulta que consume productos: el sistema inserta la atención en MongoDB y descuenta unidades en Redis. Para reducir el riesgo de inconsistencias, primero se valida el stock disponible, luego se inserta la consulta y finalmente se descuenta con un script Lua atómico. Aun así, si hubiera una falla justo entre ambos motores, podría existir una inconsistencia temporal.

Una mejora posible para un sistema productivo sería implementar un patrón de *outbox* o una saga. Por ejemplo, se podría insertar en MongoDB la consulta junto con un evento pendiente de descuento, y luego un worker aplicaría el cambio en Redis con reintentos y registro de estado. Para el alcance del TP se priorizó una implementación más directa, pero dejando explícita la limitación de coordinación entre motores.

Otra limitación es que la caché es *best-effort*. Las claves `cache:*` tienen TTL corto y se invalidan cuando la aplicación registra una nueva consulta, pero podrían quedar datos levemente desactualizados si alguien modifica MongoDB por fuera de la API. Esto es aceptable para consultas de dashboard, porque no se usa la caché como fuente de verdad ni para decisiones transaccionales.

Redis se usa como almacenamiento operativo del stock. En este setup corre dentro de Docker con la configuración simple del proyecto. Para producción habría que endurecer su persistencia y tolerancia a fallos, por ejemplo configurando AOF o snapshots RDB según la necesidad, replicación, backups y una política clara de recuperación ante caídas.

También se simplificó la seguridad del entorno. MongoDB y Redis se levantan sin autenticación ni TLS para facilitar la corrección y la ejecución local del trabajo. En un ambiente real deberían usarse credenciales, secretos fuera del repositorio, redes internas, firewall y conexiones seguras.

Por último, las fechas relativas dependen de la fecha del sistema. Esto afecta especialmente queries como últimos 60 días, vacunas vencidas, último año e ingresos del mes actual. Para que la Query 11 no quede vacía al ejecutar el TP en distintos meses, el seed agrega algunas consultas del mes corriente. Esta decisión mejora la reproducibilidad de la demo, pero también implica que parte del dataset se calcula al momento de cargar la base.

8 Conclusiones

El trabajo permitió aplicar una arquitectura de persistencia polígota con dos motores NoSQL de paradigmas distintos y responsabilidades bien separadas. MongoDB funcionó como base principal del dominio clínico y administrativo, mientras que Redis se usó para stock operativo, decrementos atómicos y caché de consultas frecuentes.

El modelo de datos se diseñó a partir de las consultas pedidas. Las entidades principales se mantuvieron como colecciones independientes y relacionadas por identificadores, lo que permitió resolver los cruces necesarios con pipelines de agregación. Redis, en cambio, se utilizó solamente donde aportaba una ventaja concreta: ordenar y filtrar stock por cantidad, descontar unidades de forma atómica y acelerar lecturas repetidas.

La solución final cumple con las 15 consultas y servicios de la consigna, mantiene el código organizado en módulos separados y deja documentadas las decisiones de diseño principales. Si bien existen limitaciones propias de coordinar dos motores sin transacción global, la implementación resulta adecuada para el alcance del TP: es simple de levantar, reproducible, fácil de probar y coherente con el uso de MongoDB y Redis dentro del sistema VetSalud.

9 Referencias

- Documentación oficial de MongoDB: <https://www.mongodb.com/docs/>
- Documentación oficial de Redis: <https://redis.io/docs/latest/>
- Documentación de Docker: <https://docs.docker.com/>